

FARMING SIMULATOR

By: Arianna Childs, Mackenzie Mann, Halden Kerzner

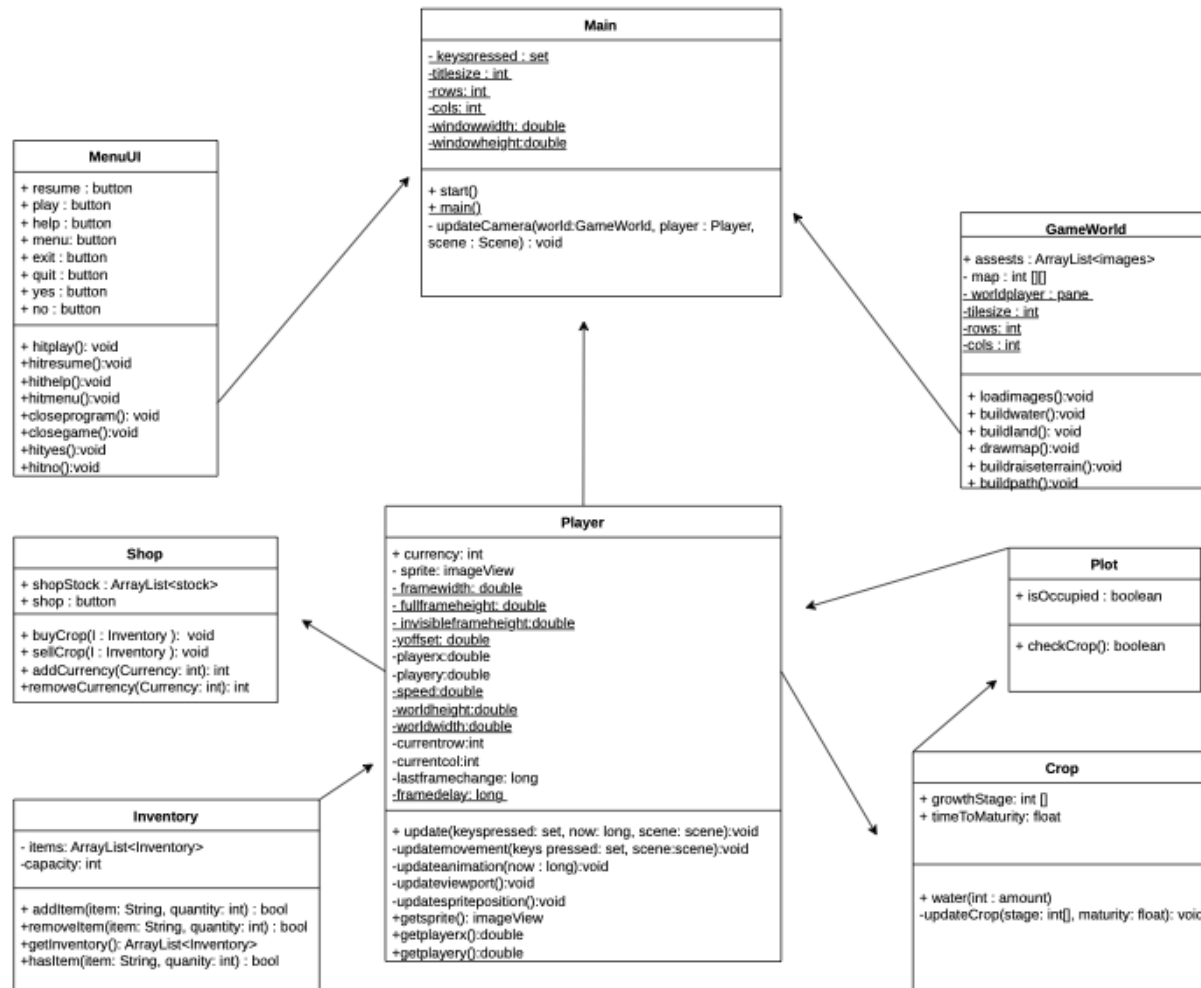


Game Description

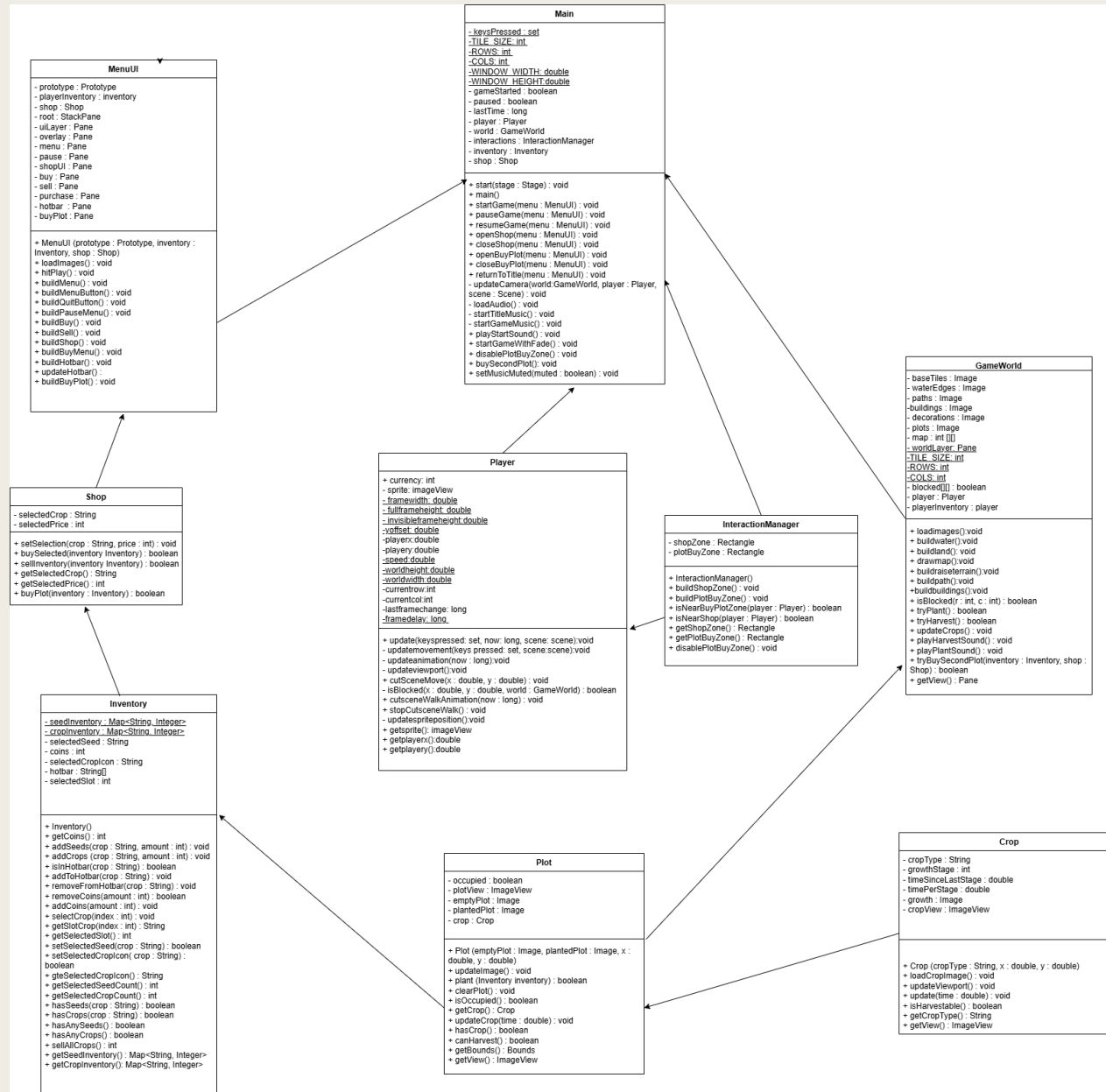


- Users play as a character that can buy/plant their own crops and manage their own farm.
- The user will have the ability to plant a particular crop, out of the 4 provided and grow it.
- Once crops are fully grown, they are put into the inventory, where they can be sold in the shop or used later by the user.
- The intended goal of the stimulator is to allow a user of any age to relax and play a fun game
- For those who like farming or managing, this is very well suited for them!

ORIGINAL UML DIAGRAM



FINAL UML DIAGRAM



```

private void updateMovement(Set<KeyCode> keysPressed, Scene scene, GameWorld world) {
    velocityX = 0;
    velocityY = 0;

    if (keysPressed.contains(KeyCode.W)) {
        velocityY = -speed;
        currentRow = 3;
    } else if (keysPressed.contains(KeyCode.S)) {
        velocityY = speed;
        currentRow = 0;
    } else if (keysPressed.contains(KeyCode.A)) {
        velocityX = -speed;
        currentRow = 1;
    } else if (keysPressed.contains(KeyCode.D)) {
        velocityX = speed;
        currentRow = 2;
    }

    double nextX = playerX + velocityX;
    double nextY = playerY + velocityY;

    // check X movement first
    if (!isBlocked(nextX, playerY, world)) {
        playerX = nextX;
    }

    // check Y movement second
    if (!isBlocked(playerX, nextY, world)) {
        playerY = nextY;
    }

    double spriteWidth = sprite.getFitWidth();
    double spriteHeight = sprite.getFitHeight();

    if (playerX < 0) {
        playerX = 0;
    }
    if (playerY < 0) {
        playerY = 0;
    }
    if (playerX > WORLD_WIDTH - spriteWidth) {
        playerX = WORLD_WIDTH - spriteWidth;
    }
    if (playerY > WORLD_HEIGHT - spriteHeight) {
        playerY = WORLD_HEIGHT - spriteHeight;
    }
}

```

```

public void updateViewport() {
    double frameWidth = growth.getWidth() / 4.0;
    double frameHeight = growth.getHeight();

    cropView.setViewport(new Rectangle2D(
        growthStage * frameWidth,
        0,
        frameWidth,
        frameHeight));
}

public void update(double time)
{
    if (growthStage >= 3) {
        return;
    }

    timeSinceLastStage += time;

    if (timeSinceLastStage >= timePerStage) {
        growthStage++;
        timeSinceLastStage = 0;
        updateViewport();
    }
}

```

```

AnimationTimer timer = new AnimationTimer() {
    @Override
    public void handle(long now) {
        if (lastTime == 0) {
            lastTime = now;
            return;
        }

        double time = (now - lastTime) / 1_000_000_000.0;
        lastTime = now;

        if (gameStarted && !paused) {
            if (cutscenePlaying) {
                keysPressed.clear();
                player.cutsceneMove(0, cutsceneSpeed);
                player.cutsceneWalkAnimation(now);
            }
            else {
                player.update(keysPressed, now, scene, world);
            }
            world.updateCrops(time);
        }

        updateCamera(world, player, scene);
    }
};

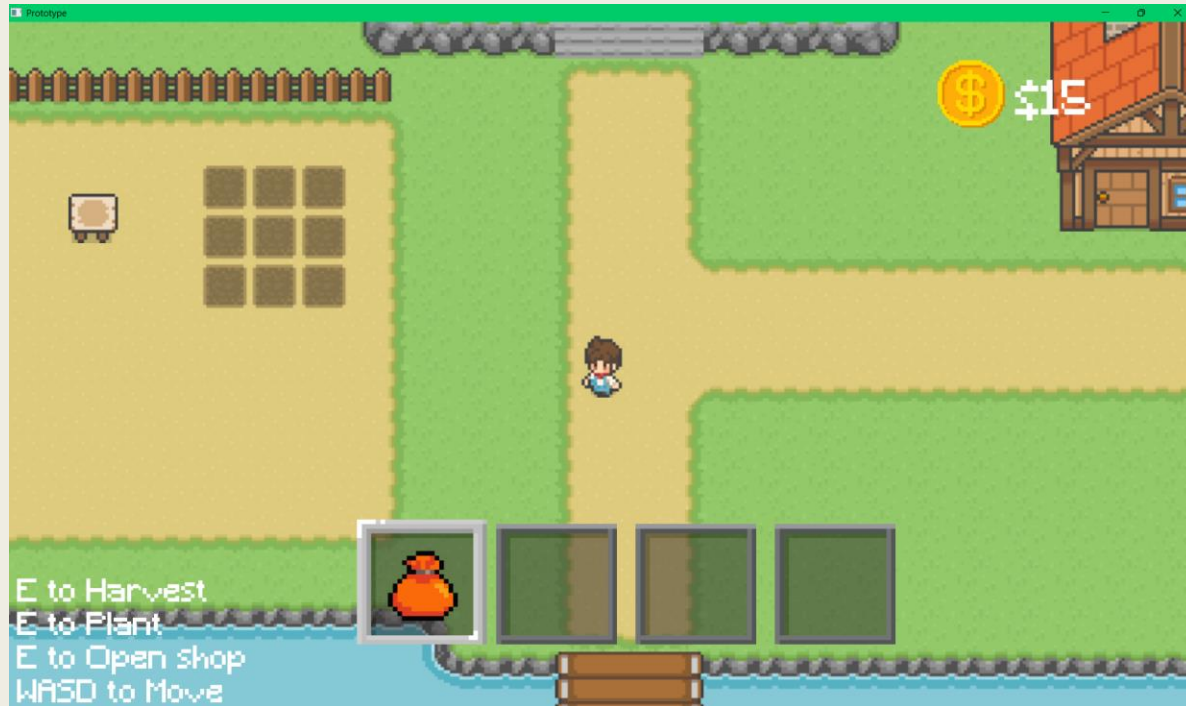
timer.start();

```

CODE SNIPPETS



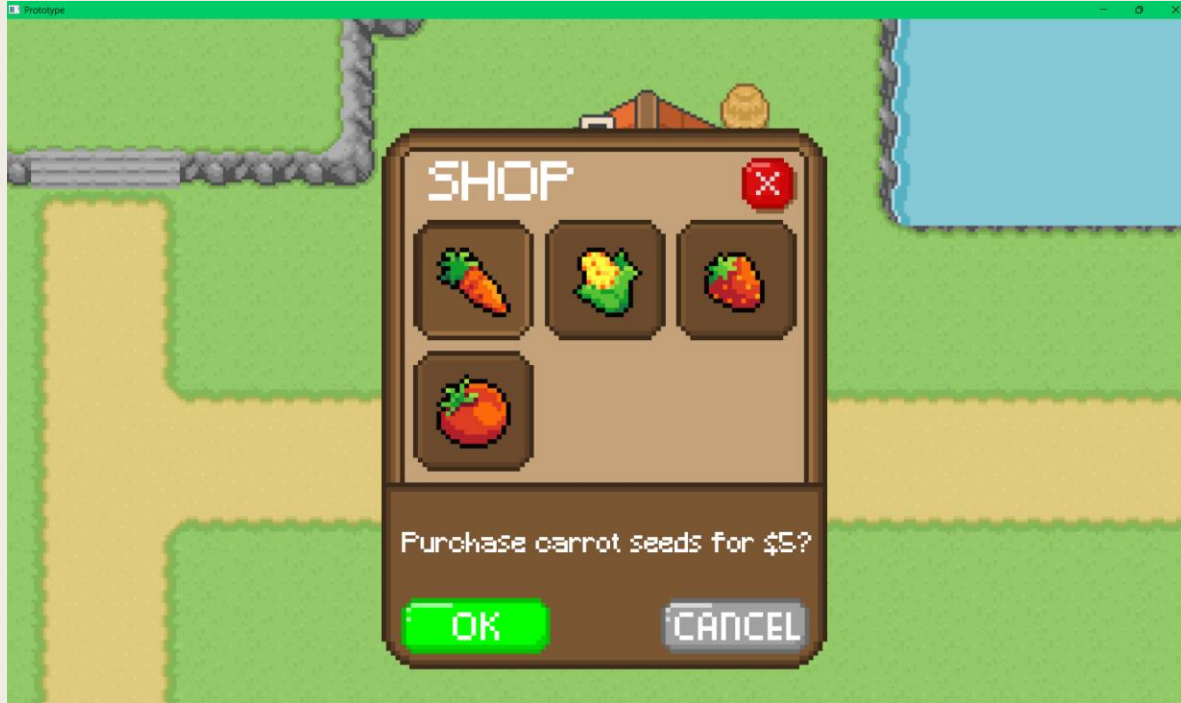
TITLE
SCREEN



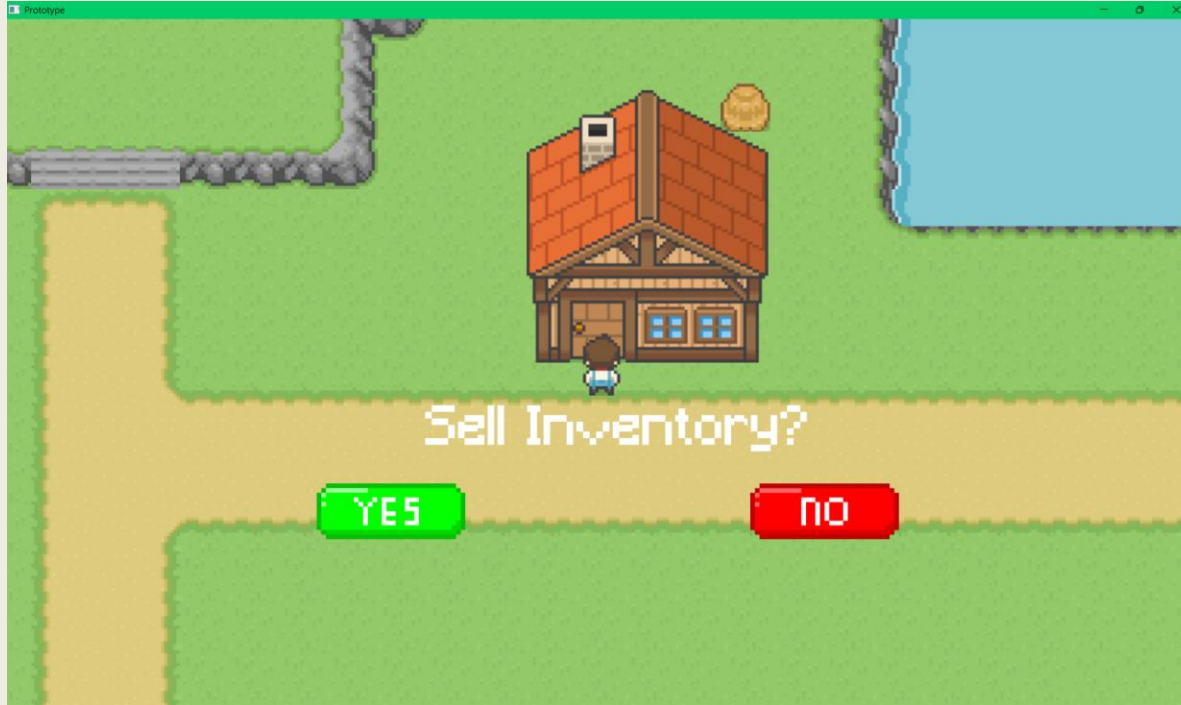
MAP



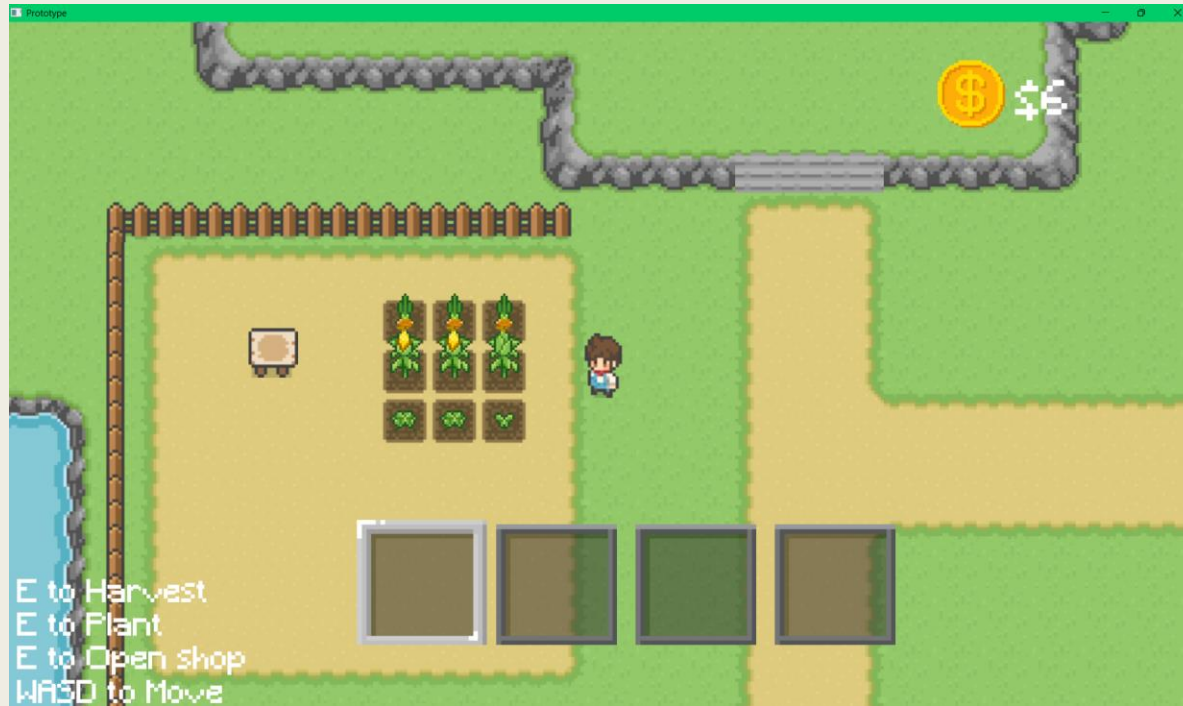
SHOP UI



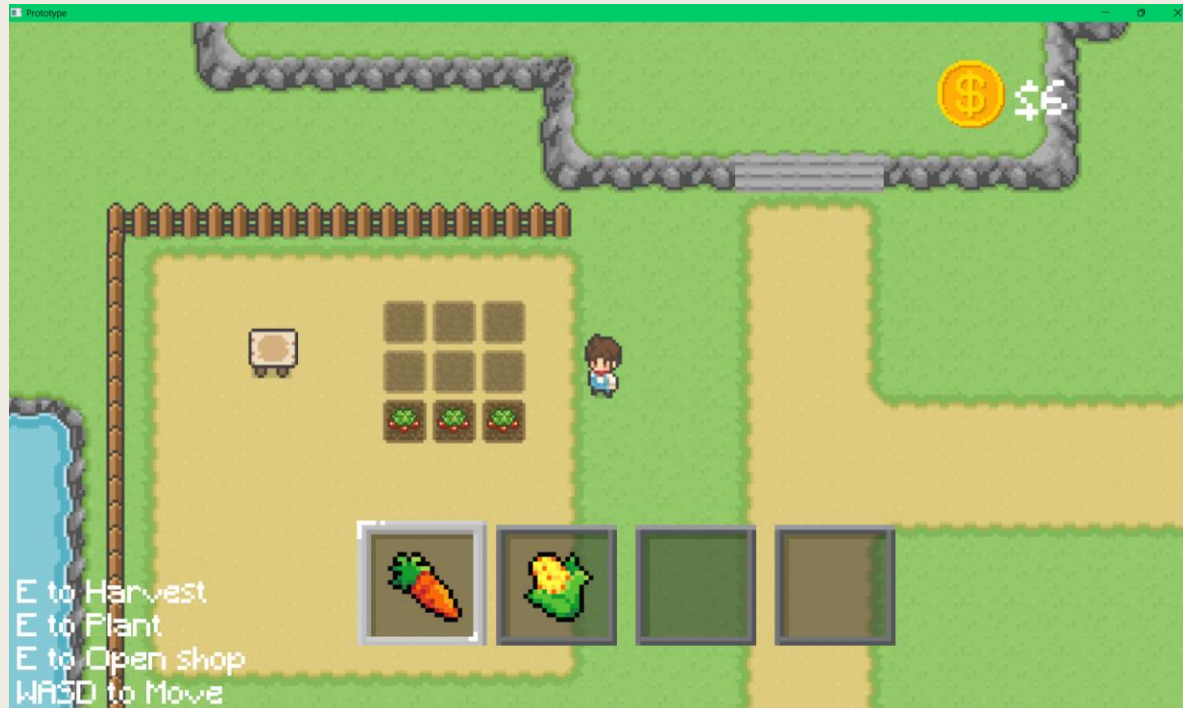
BUYING CROPS



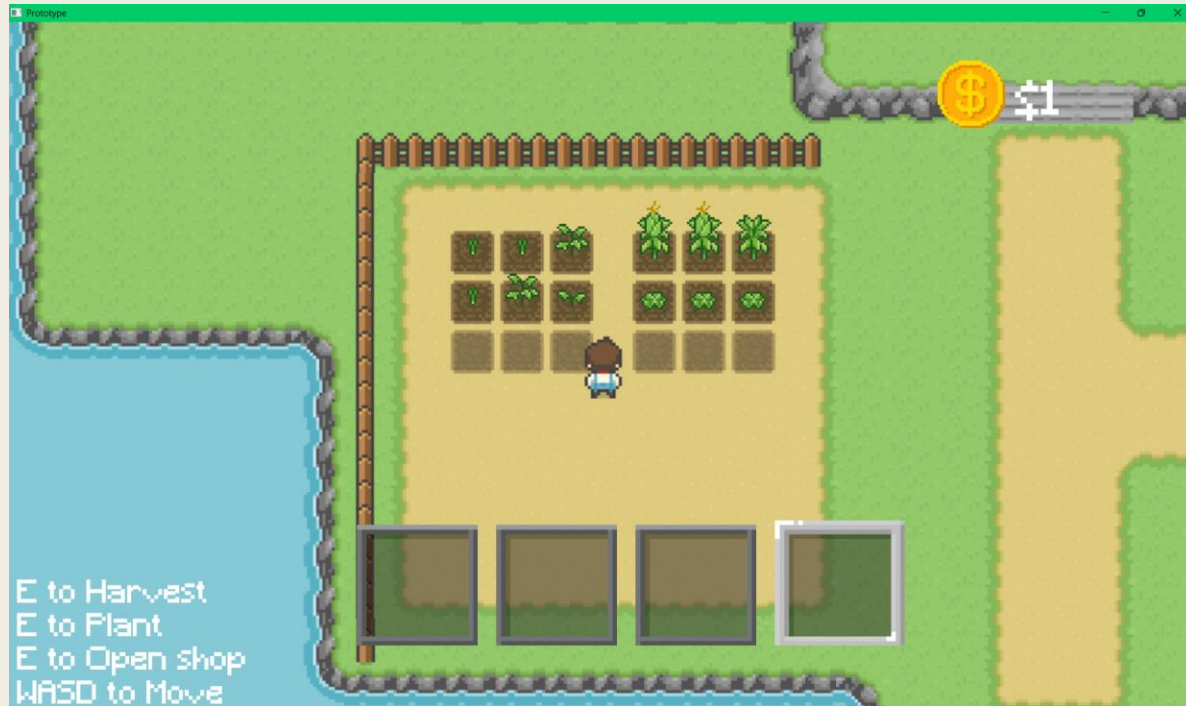
SELLING CROPS



PLANTING



HARVESTING



EXPANDING FARM

Implementation / Troubleshooting

- Collision and World Boundaries
 - Challenge: Player was able to walk through water, cliffs, houses, fences, and bridges.
 - Solution: Used blocked tile logic and collision detection integrated into the player movement.
- Inventory/Crop/Plot/Hotbar Integration
 - Challenge: Connecting planting, crop growth, harvesting, hotbar updates, and selling across multiple classes.
 - Solution: Restructured the relationships between Plot, Crop, Inventory, and GameWorld.
- Audio and Scene Transition
 - Challenge: Implementing title music, gameplay music, fade transitions, sound effects, and intro cutscene timing.
 - Solution: Used timer within the fade transition and animation timer to properly sync music change and game scene change.
- Second Plot Unlock
 - Challenge: Adding unlockable expansion while preserving planting and harvesting logic.
 - Solution: Added progression state logic, plot reveal system and reused original planting and harvesting logic.
- Animation and Movement
 - Challenge: Managing overlapping interfaces (pause menu, shop menu, settings menu) while preventing conflicts with inputs and preserving game state.
 - Solution: Implemented game state checks and menu logic to properly coordinate pausing, player input, and UI interactions.

Demonstration!

Gameplay video:

<https://youtu.be/VxVJRR-t04o?si=eaK52YqM0r6n3XRP>